



Eagles Racing Team

Relazione di Progetto

Corso “Intelligenza Artificiale: Metodi ed Applicazioni”

Anno Accademico 2025/2026

A Cura di

Ametrano Antonio - 0612710145

Iuzzolino Maria - 0612800529

1. Introduzione e Obiettivi

Il **progetto** mira a sviluppare un **agente di guida autonoma** all'interno del simulatore TORCS, con l'obiettivo di **completare un giro del circuito Corkscrew** con la vettura di **Formula 1** nel minor tempo possibile e senza uscite di pista (**hot-lap standing start**).

SVILUPPO SU 4 FASI INCREMENTALI:

1. **Rule-Based Driving**
2. **Behavioral Cloning**
3. **Fine-Tuning con Reinforcement Learning**
4. **SAC + Cross-Entropy Method**

Ogni fase è stata vincolata a **non peggiorare il risultato prodotto dalla precedente**: il nuovo **driver viene promosso a driver principale** se e solo batte le **performance** di quello corrente, in termini di tempo sul giro e stabilità.

2. Strategie di Sviluppo

Fase 1: Rule-Based Driving

In questa prima fase, il driver è un **Sistema Esperto** nel senso della “tesi debole” dell'IA.

1. **Volante controllato proporzionalmente ad angolo e posizione** in pista
2. **Curvature** stimate dall'asimmetria dei *Rangefinder*
3. **Velocità di punta** dettata da un **modello fisico** di frenata
4. **Acceleratore e freno controllati proporzionalmente**

Tale scelta nasce dal bisogno di un driver di baseline fisicamente fondato, verificabile, senza training, e per generare telemetria iniziale fin da subito.

Fase 2: Behavioral Cloning (Supervised Imitation Learning)

La rete **BCPolicy** è un **Multi-Layer Perceptron** a 2 Hidden Layers (26→128→64, ReLU) con quattro teste di output:

1. **Sterzo**
2. **Acceleratore**
3. **Freno**
4. **Marcia**

La predizione della marcia è stata poi sostituita da un algoritmo, esclusivamente basato sul numero di giri motore.

L'addestramento è stato guidato dalla discesa del gradiente (Adam, Adaptive Moment Estimation), minimizzando la **Loss MSE** (Errore Quadratico Medio) tra l'azione predetta e quella del dimostratore esperto.

Il sistema ha attraversato 2 generazioni:

1. **Blend di 2 Reti Specializzate per Rettilineo e Curva**
 1. *Selezionate secondo una soglia sul sensore frontale track[9] ed addestrate su dimostrazioni eterogenee*
2. **Singola BCPolicy [Driver promosso in questa fase]**
 1. *Addestrata per imitazione diretta del Bot interno a TORCS “Tita”*
 2. *Con telemetria estratta tramite l'implementazione di un DLL proxy, che inoltra le callback dell'interfaccia al binario originale senza alterarne la logica (riferimento a data_collection/tita/README.md)*

Praticamente un “Man-in-the-Middle” trasparente: permette di osservare tutto ciò che il Bot vede e fa, quindi raccogliere campioni per BC, senza inferire sul suo comportamento originale.

Il passaggio a singolo modello è stato motivato dalla disponibilità di un dimostratore più forte e coerente, che ha eliminato il blend (delle 2 reti specializzate) e ha permesso di ridurre STEER_GAIN da 1.8 a 1.0 (siccome un Gain più alto induce oscillazioni).

Per contrastare il problema del **Covariate Shift** (tipico del controllo sequenziale, dove l'azione predetta può indurre ad uno stato successivo fuori traiettoria, facendo sì che l'errore si propaghi fino a spostare il driver in stati mai visti prima) è stato applicato il **Dataset Aggregation (DAGger)**, in 2 varianti:

1. CICLO CLASSICO

1. Sistema Esperto della Fase 1 viene interrogato sugli stati visitati durante il rollout del BC

2. VARIANTE “SAFETY-NET”

1. Non un DAGger in senso stretto, ma un **meccanismo che passa il controllo al driver basato esclusivamente su BC (versione precedente, più sicura) se $|trackPos| > 0,55$** registrando poi le azioni di recupero intraprese come correzioni

Applicato in 8 round iterativi, l'ultimo Driver risultante è stato promosso a principale.

Fase 3: Reinforcement Learning (con Soft Actor-Critic)

È stata prediletta la strategia **SAC, Actor-Critic off-policy con ottimizzazione congiunta di Reward ed entropia**.

La scelta è stata motivata da:

1. Efficienza Campionaria Off-Policy

1. Ogni episodio raccoglie circa 5.800 step, e va anche considerato l'overhead di rilancio del simulatore TORCS

2. Esplorazione tramite Entropia Auto-Regolata

1. Lo schema canonico (warm-start dell'Actor a partire dai pesi BC, con fine-tuning completo) è stato tentato per primo, ma il prodotto si è rivelato instabile
2. Quindi l'approccio è stato sostituito da **un'architettura residuale**:
 1. **BC resta congelato, mentre una rete SAC separata apprende una correzione limitata**

Inoltre, è stata valutata l'applicazione della **PCA (Principal Component Analysis)** per i **19 Rangefinder track[]** (sensori adiacenti correlati tra loro), ma scartata: non sussiste un problema di dimensionalità con sole 26 feature in un MLP, oltre il fatto che diversi meccanismi dipendono da sensori singoli interpretabili.

Fase 4: Ottimizzazione Black-Box (Cross-Entropy Method)

Il **CEM** è un **metodo di ottimizzazione black-box a popolazione**:

1. Ad ogni generazione si **campiona un insieme di candidati (vettori di pesi)** (da una gaussiana di media μ e deviazione standard σ)
2. Ciascun candidato viene valutato per *fitness*, definita dal tempo ottenuto sul giro
3. Candidati migliori (*elite, la frazione superiore della popolazione*) vengono usati per aggiornare μ (media degli elite) e ridurre σ (iterando fino a convergenza)

Applicato in 5 round progressivi, il CEM ha prodotto il record assoluto del progetto, quindi promosso a driver principale e finale.

3. Risultati Ottenuti tra le Varie Fasi

Driver	Fase	Tempo sul Giro
Rule-Based Driver	1	148.44 s
Driver BC (Blend tra Rettilineo e Curva)	2A	121.97 s
Driver BC (con Tita Esperto Dimostratore)	2B	111.98 s
SAC Diretto (con Coefficiente di Entropia Fisso)	3A	143.24 s
SAC Diretto (Migliore tra 9 Varianti Iterative)	3B	129.68 s
SAC Residuale (Driver BC + Correzioni)	3C	127.07 s
CEM (5 ^a Versione Testata)	4	105.81 s

Tutti i driver promossi registrano 0% di uscite dalla pista.

4. Esperimenti Falliti e Cambi di Strategia

1. SAC DIRETTO E FINE-TUNING COMPLETO (SECONDO LO SCHEMA CANONICO)

- Con il coefficiente di entropia auto-regolato la policy è collassata (fuori pista)
 - Causa del problema: Reward Hacking

Con piena autorità, la policy rinuncia a un giro completo pur di massimizzare il Reward per-step accumulato.

CAMBIO DI STRATEGIA POST-ESPERIENZA: Architettura residuale, che impedisce all'Actor di allontanarsi troppo da una policy nota e sicura.

2. SAC RESIDUALE

- La strategia non ha portato ad un miglioramento del tempo sul giro

CAMBIO DI STRATEGIA POST-ESPERIENZA: Re-Training con DAgger.

3. RETRAINING UNIFICATO DEL DRIVER BC CON DATI DAGGER CONTAMINATI

- È stato tentato un Re-Training completo da zero, senza però definire pesi tra campioni derivanti da driver Rule-Based e il dataset DAgger filtrato

“Mescolare” senza pesi i campioni di un esperto lento e uno veloce causa una regressione verso la media dei due comportamenti.

CAMBIO DI STRATEGIA POST-ESPERIENZA: Fine-tuning conservativo (poche epoche, learning rate basso) invece di retraining da zero, e, per la generazione successiva, un oracolo di correzione coerente con lo stile del driver da migliorare (BC anziché driver Rule-Based).

5. Conclusioni e Sviluppi Futuri

Il driver principale soddisfa i vincoli di sicurezza e batte tutte le versioni precedenti sul tempo.

Il gap residuo tra il Driver BC e il Bot nativo Tita, che ha ricoperto il ruolo d'esperto dimostratore, è attribuibile al **limite strutturale del Behavioral Cloning** e alla rappresentazione più povera (26 feature) rispetto alla logica interna del Bot originale.

Possibili sviluppi futuri possono essere:

1. Con Behavioral Cloning

1. Proseguire i **round DAgger su Driver BC basato su Tita** (interrotti per assenza di miglioramento dopo due round)

2. Con Reinforcement Learning

1. Definizione di **Reward basato sul tempo di giro parziale**, frammentando la pista in settori, anziché basandosi sulla velocità per-step.